

UNISENAI CAMPUS JOINVILLE
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
GERÊNCIA DE CONFIGURAÇÃO E MANUTENÇÃO DE SOFTWARE

Atividade EAD — Gerência de Configuração e Manutenção de Software

ANA PAULA DOS REIS

Disciplina: Gerência de Configuração e Manutenção de Software
Professores: Emerson Amancio e Tiago Ricaldi

Joinville
2026

SUMÁRIO

1 INTRODUÇÃO AO TEMA

- 1.1 Conceito de Gerência de Configuração de Software
- 1.2 Conceito de Manutenção de Software
- 1.3 Importância no Ciclo de Vida do Software

2 PRINCIPAIS ÁREAS ENVOLVIDAS

- 2.1 Controle de Versão e Versionamento de Software
- 2.2 Gerenciamento de Mudanças
- 2.3 Controle de Artefatos
- 2.4 Baselines e Rastreabilidade
- 2.5 Tipos de Manutenção de Software
- 2.6 Boas Práticas de Organização de Repositórios
- 2.7 Padronização de Commits e Branches

3 APLICAÇÃO PRÁTICA

4 EXEMPLOS E ESTUDO DE CASO

5 CONCLUSÃO

REFERÊNCIAS

1 INTRODUÇÃO AO TEMA

1.1 Conceito de Gerência de Configuração de Software

A Gerência de Configuração de Software (GCS) é um conjunto de atividades desenvolvidas para administrar as mudanças ao longo de todo o ciclo de vida do software. De acordo com Pressman e Maxim (2021, p. 470), a GCS é uma atividade de garantia da qualidade do software que é aplicada durante o processo de software inteiro. A norma IEEE Std 828-2012 define a GCS como a disciplina de identificar a configuração de um sistema em pontos discretos no tempo, com o propósito de sistematicamente controlar as mudanças na configuração e manter a integridade e rastreabilidade da configuração durante todo o ciclo de vida do sistema.

Em termos práticos, a GCS abrange o controle de versões de código-fonte, documentação, requisitos, arquivos de configuração e qualquer outro artefato que compõe o produto de software. A ausência de uma GCS bem estruturada resulta em caos: desenvolvedores sobrescrevendo o trabalho uns dos outros, versões incompatíveis sendo implantadas em produção e ausência de histórico auditável das decisões técnicas tomadas ao longo do tempo.

Harmel-Law (2024) apresenta uma perspectiva contemporânea e inovadora sobre o controle de artefatos de decisão arquitetural, ao propor os Registros de Decisão Arquitetural (ADRs — *Architectural Decision Records*). O autor argumenta que os ADRs são documentos curtos que costumam seguir um formato típico, e que cada ADR registra uma única decisão arquitetural tomada, junto das informações de suporte apropriadas (HARMEL-LAW, 2024, p. 116). Essa abordagem representa uma extensão natural da GCS tradicional: não apenas o código é versionado, mas também as decisões que o moldaram.

1.2 Conceito de Manutenção de Software

A Manutenção de Software consiste no conjunto de atividades realizadas após a entrega de um produto de software com o objetivo de corrigir defeitos, melhorar desempenho ou adaptar o produto a um ambiente modificado. Sommerville (2019, p. 241) define manutenção de software como o processo de modificar um programa depois que ele foi entregue para uso.

Do ponto de vista econômico, a manutenção representa a fase mais longa e custosa do ciclo de vida de um software. Estima-se que entre 60% e 80% do custo total de um sistema seja destinado às atividades de manutenção (PRESSMAN; MAXIM, 2021). Esse dado reforça a

necessidade de práticas robustas de GCS: sem rastreabilidade e controle adequados, cada intervenção de manutenção torna-se uma operação de risco elevado.

1.3 Importância no Ciclo de Vida do Software

A GCS e a manutenção de software são complementares e interdependentes. A GCS fornece a infraestrutura de controle que torna a manutenção segura, rastreável e eficiente. Sem GCS, a manutenção opera às cegas; sem processos de manutenção claros, a GCS torna-se um repositório estático sem valor operacional.

Harmel-Law (2024) demonstra essa interdependência ao discutir como decisões arquiteturais precisam ser registradas, rastreadas e eventualmente substituídas ao longo da vida de um sistema. O autor introduz o conceito de status dos ADRs — Rascunho (*Draft*), Proposto (*Proposed*), Aceito (*Accepted*) e Substituído (*Superseded*) —, que funciona como um mecanismo de baseline e rastreabilidade análogo ao versionamento de código-fonte. Segundo o autor, os ADRs são imutáveis: escritos uma vez, lidos muitas vezes, e nunca atualizados após serem aceitos (HARMEL-LAW, 2024, p. 116), refletindo o princípio fundamental da imutabilidade dos registros históricos na GCS.

2 PRINCIPAIS ÁREAS ENVOLVIDAS

2.1 Controle de Versão e Versionamento de Software

O controle de versão é o alicerce técnico da GCS moderna. Por meio de Sistemas de Controle de Versão (VCS — *Version Control Systems*), cada alteração em artefatos de software é registrada com metadados como autor, data, hora e descrição da mudança. Os VCS modernos, especialmente o Git, adotam um modelo distribuído em que cada desenvolvedor possui uma cópia completa do histórico do repositório, eliminando o ponto único de falha presente nos modelos centralizados mais antigos.

Harmel-Law (2024) evidencia a relevância do controle de versão distribuído ao descrever, no relato de adoção do processo de conselho arquitetural (*Architecture Advice Process*), que as equipes que adotam entrega baseada em tronco (*trunk-based delivery*) eliminam *pull requests* de longa duração, melhorando métricas de frequência de implantação e tempo de entrega. O autor relata o caso de uma equipe que, ao estudar a obra *Accelerate*, percebeu que migrar para uma prática de entrega baseada em tronco permitiria eliminar completamente os *pull requests* e, conseqüentemente, melhorar suas métricas de lead time e frequência de implantação, fazendo

isso de forma segura (HARMEL-LAW, 2024, p. 78). Esse contexto demonstra que o controle de versão não é apenas uma ferramenta técnica, mas um habilitador de práticas ágeis e de qualidade.

Quadro 1 — Comparativo entre modelos de controle de versão

Aspecto	VCS Centralizado (SVN)	VCS Distribuído (Git)
Modelo de repositório	Um único servidor central	Cada cópia é um repositório completo
Trabalho offline	Limitado — requer conexão para a maioria das operações	Pleno — commits, branches e histórico locais
Branching	Pesado e custoso	Leve e incentivado
Ponto único de falha	Sim	Não
Rastreabilidade	Linear e centralizada	Distribuída com grafos de commits

Fonte: Elaboração própria (2026).

2.2 Gerenciamento de Mudanças

O Gerenciamento de Mudanças (*Change Management*) é o processo formal pelo qual solicitações de alteração são avaliadas, aprovadas, implementadas e verificadas. Segundo Pressman e Maxim (2021, p. 475), o controle de mudanças combina procedimentos humanos e ferramentas automatizadas para proporcionar um mecanismo de controle das mudanças.

Harmel-Law (2024) propõe uma reinterpretação radical do gerenciamento de mudanças arquiteturais por meio do *Architecture Advice Process*. Em vez de centralizar a aprovação de mudanças em um único arquiteto ou comitê, o processo distribui o poder de decidir para qualquer membro da equipe, exigindo apenas que o proponente da mudança consulte dois grupos: as partes afetadas pela decisão e os especialistas no domínio da mudança. O autor sintetiza esse princípio da seguinte forma:

Qualquer pessoa pode tomar (selecionar uma opção de decisão) e comunicar uma decisão arquitetural, desde que, durante a etapa de formulação da opção, busque a orientação de todos que serão significativamente afetados pela decisão e de pessoas que possuam expertise na área em que a decisão está sendo tomada (HARMEL-LAW, 2024, p. 71, tradução nossa).

Essa abordagem descentralizada do gerenciamento de mudanças mantém a rastreabilidade e o controle sem criar os gargalos típicos dos modelos tradicionais, nos quais um único arquiteto torna-se ponto de bloqueio para toda a equipe.

2.3 Controle de Artefatos

O controle de artefatos abrange a identificação, armazenamento, versionamento e recuperação de todos os produtos de trabalho gerados durante o desenvolvimento de software: código-fonte, documentação de requisitos, casos de teste, scripts de implantação, diagramas de arquitetura, arquivos de configuração de infraestrutura e registros de decisão.

Harmel-Law (2024) amplia a concepção tradicional de artefatos ao incluir explicitamente as decisões arquiteturais como itens sujeitos a controle formal. Os ADRs, na proposta do autor, possuem identificadores únicos, metadados de autoria e data, ciclo de vida controlado por status e imutabilidade após aceitação. O autor recomenda que os ADRs sejam armazenados em ferramentas de desenvolvimento como wikis ou repositórios de controle de código-fonte, de modo que sejam escritos em formatos familiares aos desenvolvedores e armazenados nas próprias ferramentas de desenvolvimento (HARMEL-LAW, 2024, p. 154).

Quadro 2 — Tipos de artefatos e mecanismos de controle recomendados

Tipo de Artefato	Ferramenta de Controle	Observações
Código-fonte	Git (GitHub, GitLab, Bitbucket)	Versionamento completo com histórico
Documentação	Git + wiki ou Confluence	Versionamento junto ao código
Requisitos	Jira, Azure DevOps, GitLab Issues	Rastreabilidade até o código
Decisões arquiteturais (ADRs)	Git ou wiki do projeto	Imutáveis após aceitos (HARMEL-LAW, 2024)
Artefatos de build	Nexus, Artifactory, GitHub Packages	Versionamento semântico
Infraestrutura como código	Git + Terraform, Ansible	Mesma rastreabilidade do código

Fonte: Elaboração própria (2026).

2.4 Baselines e Rastreabilidade

Uma *baseline* é uma fotografia formal e aprovada de um conjunto de itens de configuração em um momento específico do projeto, que serve como base para o desenvolvimento subsequente e pode ser modificada apenas por meio do processo formal de controle de mudanças. As baselines são fundamentais para a rastreabilidade, pois permitem relacionar requisitos, código, testes e documentação de forma coerente e auditável.

A abordagem de ADRs proposta por Harmel-Law (2024) implementa um mecanismo de baseline de decisões arquiteturais. Cada ADR com status *Accepted* representa uma baseline de uma decisão específica. Quando uma decisão precisa ser revisada, um novo ADR é criado e o anterior recebe o status *Superseded*, mantendo o histórico íntegro. Conforme o autor explica,

os ADRs são imutáveis: uma vez decididos, passam a existir no registro histórico da arquitetura, e não faria sentido voltar no tempo para alterar esse registro (HARMEL-LAW, 2024, p. 151). Esse princípio espelha diretamente a lógica das baselines na GCS clássica.

Quadro 3 — Ciclo de vida de um ADR e analogia com GCS tradicional

Status do ADR	Descrição	Analogia em GCS
Draft	Decisão em elaboração, sem efeito formal	Item de configuração em desenvolvimento
Proposed	Decisão proposta, aberta para consulta de conselho	Solicitação de mudança em análise
Accepted	Decisão formalmente aceita e implementada	Baseline aprovada
Superseded	Decisão substituída por um ADR posterior	Versão obsoleta com histórico preservado

Fonte: Adaptado de Harmel-Law (2024, p. 150-151).

2.5 Tipos de Manutenção de Software

A classificação dos tipos de manutenção de software é amplamente aceita e baseia-se nos trabalhos de Swanson (1976), posteriormente incorporados à norma ISO/IEC 14764:2006. Quatro categorias principais são reconhecidas.

2.5.1 Manutenção Corretiva

A manutenção corretiva destina-se a corrigir defeitos descobertos após a entrega do software. Envolve a identificação da causa raiz do problema, a implementação da correção e a verificação de que a correção não introduziu novos defeitos (regressão). Sommerville (2019) estima que, em média, 20% dos esforços de manutenção sejam corretivos.

2.5.2 Manutenção Adaptativa

A manutenção adaptativa consiste em modificar o software para mantê-lo utilizável em um ambiente tecnológico em constante evolução: atualizações de sistema operacional, migração de plataforma de nuvem, mudanças em APIs de terceiros ou adaptações a novas regulamentações legais. Harmel-Law (2024) ilustra esse tipo de manutenção ao descrever como decisões arquiteturais precisam ser revisitadas quando o contexto tecnológico muda, uma vez que decisões posteriores exercem forças evolutivas descendentes sobre as decisões que as sustentam (HARMEL-LAW, 2024, p. 377).

2.5.3 Manutenção Perfectiva (Evolutiva)

A manutenção perfectiva, também denominada evolutiva, visa aperfeiçoar ou adicionar funcionalidades ao software para atender a novos requisitos dos usuários ou melhorar atributos de qualidade como desempenho, usabilidade e segurança. Representa a maior fatia dos esforços de manutenção, estimada entre 50% e 60% do total (PRESSMAN; MAXIM, 2021). Harmel-Law (2024) relaciona esse tipo de manutenção à evolução contínua das arquiteturas de software, descrevendo como sistemas bem gerenciados permitem que os times construam, operem e evoluam seus sistemas (HARMEL-LAW, 2024, p. 162) de forma sustentável.

2.5.4 Manutenção Preventiva

A manutenção preventiva consiste em modificar o software para detectar e corrigir falhas latentes antes que se tornem falhas efetivas. Inclui refatoração de código para reduzir dívida técnica, atualização de dependências com vulnerabilidades conhecidas e melhoria da cobertura de testes automatizados. Harmel-Law (2024) aborda esse conceito ao discutir a importância de decisões que removem acoplamentos desnecessários e simplificam a arquitetura, citando um caso real em que uma equipe precisou refatorar um serviço de fluxo de trabalho para consolidar seus padrões de uso, tornando explicitamente mais fácil utilizá-lo para o que fazia bem, e mais difícil ou impossível utilizá-lo para o que fazia mal (HARMEL-LAW, 2024, p. 90).

Quadro 4 — Resumo dos tipos de manutenção de software

Tipo	Objetivo	Proporção Estimada	Exemplo
Corretiva	Corrigir defeitos	~20%	Bug de cálculo de imposto corrigido
Adaptativa	Adaptar ao novo ambiente	~25%	Migração de API de pagamento
Perfectiva/Evolutiva	Adicionar/melhorar funcionalidades	~50-60%	Novo módulo de relatórios
Preventiva	Evitar falhas futuras	~5%	Refatoração para reduzir dívida técnica

Fonte: Adaptado de Pressman e Maxim (2021, p. 532) e Sommerville (2019, p. 244).

2.6 Boas Práticas de Organização de Repositórios

A organização de repositórios de código-fonte é um fator crítico para a produtividade das equipes e para a eficiência da GCS. As principais boas práticas incluem:

- Estrutura de diretórios clara: separar código-fonte (/src), testes (/test), documentação (/docs) e scripts de infraestrutura (/infra ou /deploy).

- Arquivo README: manter um arquivo README.md na raiz do projeto com instruções claras de configuração, execução e contribuição.
- Gitignore efetivo: utilizar arquivos .gitignore adequados à linguagem e ao framework para evitar o versionamento de arquivos binários compilados, logs e configurações locais do desenvolvedor.
- Monorepo versus multirepo: avaliar a estratégia correta para a organização, considerando que monorepos podem facilitar a padronização e refatorações atômicas, enquanto múltiplos repositórios oferecem maior isolamento e controle de acesso independente.
- Documentação como código: manter a documentação técnica relevante versionada no mesmo repositório do código-fonte, facilitando a atualização sincronizada.

2.7 Padronização de Commits e Branches

A padronização das mensagens de commit e da estratégia de branching é fundamental para a rastreabilidade e para a automação do processo de entrega contínua (CI/CD). O padrão *Conventional Commits* define uma convenção leve que especifica que cada mensagem de commit deve iniciar com um tipo (feat, fix, docs, style, refactor, test, chore), seguido de escopo opcional e descrição obrigatória. Essa padronização habilita a geração automática de changelogs e a determinação automatizada de versões semânticas.

Para estratégias de branching, os modelos mais difundidos são o *Gitflow*, adequado a projetos com releases programados, e o *Trunk-Based Development*, adequado a equipes que praticam integração e entrega contínua. Harmel-Law (2024) menciona explicitamente o benefício da entrega baseada em tronco ao descrever que equipes que a adotam conseguem eliminar completamente os pull requests e, consequentemente, melhorar suas métricas de lead time e frequência de implantação (HARMEL-LAW, 2024, p. 78), conectando diretamente a estratégia de branching às métricas de desempenho de entrega de software (DORA Metrics).

3 APLICAÇÃO PRÁTICA

Para ilustrar a aplicação dos conceitos apresentados, será analisado um projeto hipotético de desenvolvimento de um sistema de gestão acadêmica para uma instituição de ensino de médio porte. O sistema é composto por módulos de matrícula, gestão de disciplinas, lançamento de notas, geração de relatórios e integração com sistemas financeiros externos. A equipe é formada por oito desenvolvedores organizados em duas equipes de quatro, dois analistas de requisitos e um arquiteto de soluções.

3.1 Controle de Versão Aplicado

O projeto adota Git como VCS com repositório hospedado no GitHub. A estratégia de branching utilizada é o *Trunk-Based Development* com *feature flags*, conforme recomendado por Harmel-Law (2024) para equipes que almejam alta frequência de implantação. As branches de funcionalidade têm vida útil máxima de dois dias antes de serem integradas ao tronco principal, reduzindo os riscos de conflitos de merge e garantindo integração contínua real.

Os tipos de manutenção que o sistema pode sofrer são variados: manutenção corretiva para correção de cálculos de médias; manutenção adaptativa quando o Ministério da Educação altera as normas de registro acadêmico; manutenção perfectiva para a adição de novos tipos de relatórios solicitados pelos gestores; e manutenção preventiva por meio de refatorações periódicas para reduzir a dívida técnica acumulada.

3.2 ADRs como Ferramenta de GCS

O arquiteto de soluções adota a prática de ADRs proposta por Harmel-Law (2024) para registrar as decisões arquiteturais significativas. Seguindo o modelo descrito pelo autor, cada ADR contém título, status, data, autor, decisão tomada, contexto, opções consideradas, consequências e registro do conselho recebido. Por exemplo, a decisão de adotar PostgreSQL como banco de dados principal foi registrada no ADR-001, com status *Accepted*, após consulta às equipes afetadas e a especialistas em banco de dados.

Quando surgiu a necessidade de avaliar a migração para um banco de dados orientado a documentos para o módulo de relatórios, um novo ADR-007 foi iniciado com status *Draft*, seguindo o processo de conselho arquitetural. O ADR-001 permaneceu inalterado, preservando o histórico da decisão original. Após análise e consulta, optou-se por manter o PostgreSQL, e o ADR-007 foi aceito como decisão de não migração, exemplificando que decidir não agir pode ser preferível a adiar indefinidamente a própria ideia de decisão (HARMEL-LAW, 2024, p. 316), o que também representa uma decisão válida e rastreável.

3.3 Riscos da Ausência de Gerência de Configuração

A ausência de GCS no projeto hipotético geraria riscos concretos e mensuráveis:

- Sobrescrita de código: desenvolvedores poderiam sobrescrever acidentalmente o trabalho uns dos outros ao enviar arquivos manualmente para um servidor compartilhado.

- Impossibilidade de rollback: na ocorrência de um defeito crítico em produção, a equipe não teria um mecanismo confiável para reverter o sistema ao último estado estável conhecido.
- Débito técnico invisível: sem documentação de decisões (ADRs), o conhecimento sobre o porquê de certas escolhas de design se perderia com a saída de membros da equipe, tornando a manutenção futura mais lenta e arriscada.
- Paralisia na entrega: sem uma estratégia de branching leve, o processo de merge de código se tornaria um gargalo, impedindo entregas frequentes e seguras.

4 EXEMPLOS E ESTUDO DE CASO

4.1 Estudo de Caso: Falha por Ausência de Controle de Versão

Em 2012, o banco Knight Capital Group sofreu prejuízos de aproximadamente 440 milhões de dólares em menos de uma hora devido a uma falha em seu sistema de negociação algorítmica. A investigação posterior revelou que um módulo de software legado havia sido reativado inadvertidamente durante um processo de atualização mal gerenciado. A ausência de um processo rigoroso de GCS — incluindo a falta de imutabilidade dos artefatos de build e a inexistência de procedimentos de rollback automatizados — contribuiu diretamente para a magnitude do desastre. O incidente exemplifica o conceito discutido por Harmel-Law (2024, p. 22) sobre como sistemas complexos podem perder o controle ("tip out of control") quando os artefatos de configuração não são gerenciados adequadamente.

4.2 Estudo de Caso: Sucesso com Práticas de GCS — Implantação Contínua

A Amazon realiza, em média, dezenas de milhares de implantações de software por dia em seus sistemas de produção (FORSGREN; HUMBLE; KIM, 2018). Esse nível de frequência só é possível graças a práticas maduras de GCS: controle de versão rigoroso, testes automatizados em múltiplas camadas, pipelines de CI/CD com validações automatizadas e processos de rollback automatizados. Harmel-Law (2024, p. 36) menciona a Amazon no contexto do princípio de construir e operar ("build and run"), descrevendo como o controle sobre o ciclo completo de entrega — do código à produção — é fundamental para a qualidade arquitetural.

O ponto central desse estudo de caso é que a velocidade de entrega não compromete a qualidade quando a GCS está estruturada. Ao contrário, a GCS bem implementada é o que habilita a velocidade com segurança.

4.3 Estudo de Caso Fictício: Manutenção Mal Planejada

Considere o cenário de uma startup de comércio eletrônico que desenvolveu seu sistema de pagamentos sem documentação de decisões arquiteturais e sem um processo formal de GCS. Após dois anos, a empresa decide migrar do provedor de pagamentos original para um novo, motivada por melhores condições comerciais. A equipe de desenvolvimento, agora composta majoritariamente por membros diferentes dos fundadores, não encontra documentação sobre os motivos da escolha original do provedor nem sobre as integrações realizadas.

O processo de migração, estimado inicialmente em três semanas, leva quatro meses. Sem rastreabilidade dos requisitos que motivaram integrações específicas, a equipe precisa fazer engenharia reversa do código legado. Sem ADRs, não há como saber se certas peculiaridades do código são bugs ou decisões intencionais. Sem um processo formal de GCS, cada tentativa de integração com o novo provedor é realizada diretamente no ambiente de produção, causando instabilidade para os usuários finais.

Esse cenário ilustra como a ausência de GCS transforma uma manutenção adaptativa relativamente simples em uma operação de alto risco e custo elevado. Harmel-Law (2024, p. 364) descreve situações similares como resultado de decisões inadequadas acumuladas sucessivamente umas sobre as outras, formando o que o autor denomina dívida de design ("design debt").

4.4 Integração de ADRs com o Processo de Manutenção

Harmel-Law (2024) apresenta um exemplo concreto de manutenção perfectiva gerenciada por meio de ADRs ao descrever a refatoração de um serviço de fluxo de trabalho que havia se tornado um gargalo de entrega. O arquiteto identificou três padrões de uso distintos no serviço e propôs sua divisão em decisões menores e independentes, cada uma documentada em um ADR específico. As decisões foram divididas da seguinte forma:

- ADR 1: definição de um modelo canônico de dados para padronizar a comunicação entre os componentes.
- ADR 2: separação do serviço monolítico em componentes menores e especializados.
- ADR 3: definição de novas políticas de versionamento de API para os serviços refatorados.

Essa fragmentação da manutenção em unidades menores, cada uma com seu próprio registro de decisão, exemplifica como a GCS — por meio dos ADRs — transforma atividades de manutenção complexas em processos gerenciáveis, rastreáveis e com menor risco de falha.

5 CONCLUSÃO

Este trabalho examinou os fundamentos e as práticas da Gerência de Configuração e da Manutenção de Software, demonstrando sua centralidade para a qualidade, sustentabilidade e evolução dos sistemas de software modernos. Por meio da integração entre a literatura clássica da engenharia de software — representada por Pressman e Maxim (2021) e Sommerville (2019) — e a perspectiva contemporânea oferecida por Harmel-Law (2024), foi possível construir uma visão abrangente e atualizada do tema.

A análise revelou que a GCS e a manutenção de software são processos complementares e interdependentes. A GCS fornece a infraestrutura de controle que torna a manutenção segura, rastreável e eficiente; enquanto os processos de manutenção, por sua vez, validam e aperfeiçoam continuamente a eficácia da GCS. Sem GCS adequada, cada atividade de manutenção — seja corretiva, adaptativa, perfectiva ou preventiva — torna-se uma operação de risco elevado, susceptível a regressões, perda de histórico e acúmulo acelerado de dívida técnica.

A contribuição mais relevante de Harmel-Law (2024) para este trabalho reside na proposta dos Registros de Decisão Arquitetural (ADRs) como extensão natural da GCS tradicional. Os ADRs aplicam os princípios fundamentais da GCS — identificação, controle, contabilidade de status e auditoria — às decisões arquiteturais, que historicamente eram gerenciadas de forma implícita e não documentada. O ciclo de vida dos ADRs (Draft → Proposed → Accepted → Superseded) espelha diretamente o conceito de baseline da GCS clássica, enquanto o Architecture Advice Process redesenha o gerenciamento de mudanças arquiteturais de forma descentralizada, eliminando os gargalos típicos dos modelos tradicionais sem abrir mão do controle e da rastreabilidade.

Os estudos de caso analisados confirmaram empiricamente as consequências da ausência de GCS — evidenciadas pelo desastre do Knight Capital Group e pelo cenário fictício da startup de e-commerce — e os benefícios concretos de práticas maduras, como a capacidade da Amazon de realizar dezenas de milhares de implantações diárias com elevado nível de qualidade e disponibilidade.

Para o mercado de trabalho na área de tecnologia, os conceitos abordados neste trabalho são essenciais. Profissionais que dominam GCS e manutenção de software estão preparados para trabalhar em times que entregam valor de forma contínua, segura e rastreável — uma

expectativa fundamental nas organizações que adotam práticas DevOps, entrega contínua e arquiteturas de microsserviços. A habilidade de escrever boas mensagens de commit, organizar repositórios de forma coerente, conduzir revisões de código eficazes e documentar decisões arquiteturais em ADRs são diferenciais competitivos significativos para qualquer desenvolvedor ou arquiteto de software.

Por fim, ressalta-se que a GCS e a manutenção de software não são atividades burocráticas impostas sobre o processo de desenvolvimento, mas sim habilitadoras fundamentais da agilidade, da qualidade e da longevidade dos sistemas de software. Como sintetiza Harmel-Law (2024, p. 117), boas práticas de registro e controle permitem que a organização aprenda com seu próprio histórico, tornando cada decisão futura mais informada e cada processo de manutenção mais eficiente.

REFERÊNCIAS

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. *Accelerate: the science of lean software and devops: building and scaling high performing technology organizations*. Portland: IT Revolution Press, 2018.

HARMEL-LAW, Andrew. *Facilitating software architecture: empowering teams to make architectural decisions*. Foreword by Sarah Wells. Sebastopol: O'Reilly Media, 2024.

IEEE. *IEEE Std 828-2012: IEEE Standard for Configuration Management in Systems and Software Engineering*. New York: IEEE, 2012.

ISO/IEC. *ISO/IEC 14764:2006: Software Engineering — Software Life Cycle Processes — Maintenance*. Geneva: ISO, 2006.

PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de software: uma abordagem profissional*. 9. ed. Porto Alegre: AMGH, 2021.

SOMMERVILLE, Ian. *Engenharia de software*. 10. ed. São Paulo: Pearson Education do Brasil, 2019.

SWANSON, E. Burton. The dimensions of maintenance. In: INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING, 2., 1976, San Francisco. *Proceedings* [...]. San Francisco: IEEE Computer Society Press, 1976. p. 492-497.